

DACs and Analogue Signal Generation

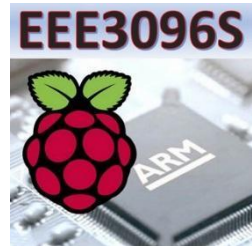
Sampling & Data Conversion · Module 4,
Lecture 4 of 4

Travimadox Webb^{1,2}

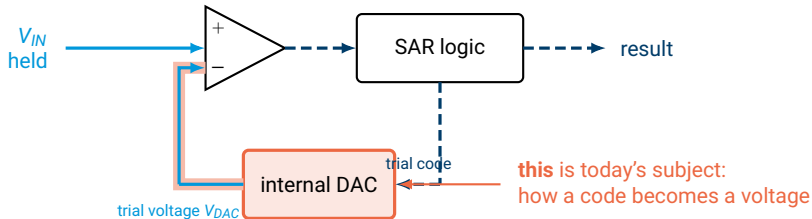
September 28, 2026

¹ Department of Electrical Engineering
University of Cape Town

² African Robotics Unit (ARU)



DACs inside the ADCs we studied

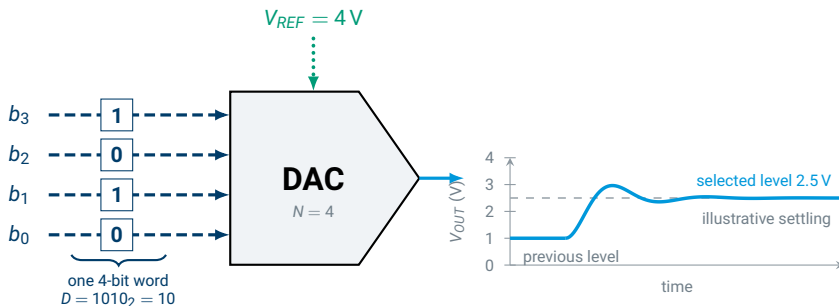


A SAR ADC uses a DAC to make each trial voltage. A multistage ADC uses one to reconstruct the coarse estimate before subtracting it.

We have already met a DAC inside an ADC. Now we examine how it works.

What is a DAC?

DAC = **D**igital-to-**A**nalogue **C**onverter: converts a digital code into a corresponding analogue voltage or current. (Our examples use voltage output.)

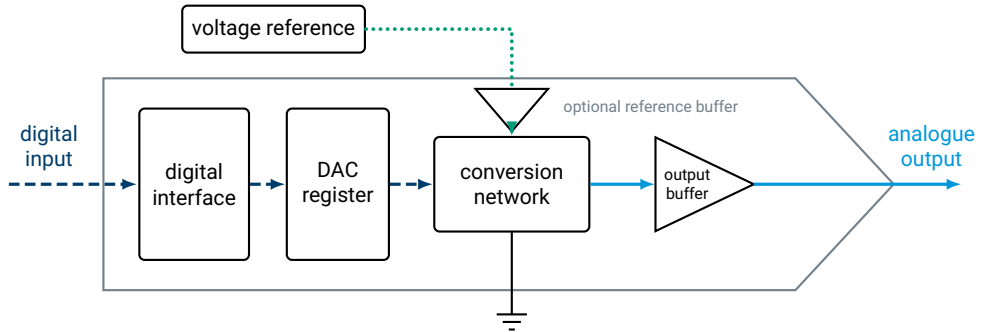


A microcontroller uses a DAC to set an analogue control level or generate a signal, adding filtering or output drive where needed.

The code selects an analogue level. Analogue describes the physical output, yet an N -bit DAC still has only 2^N nominal levels.

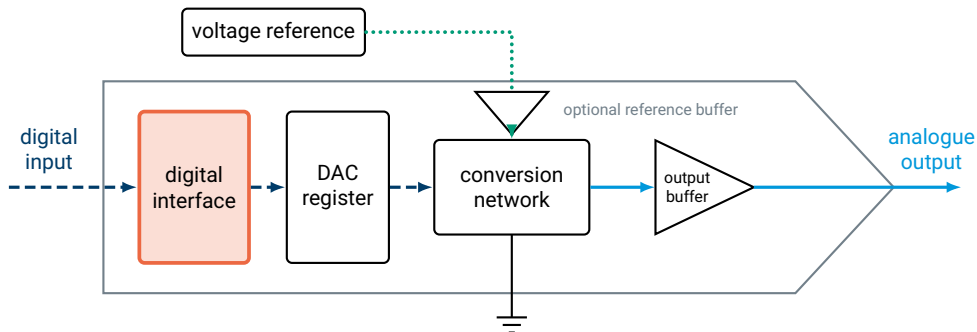


Building blocks of a DAC



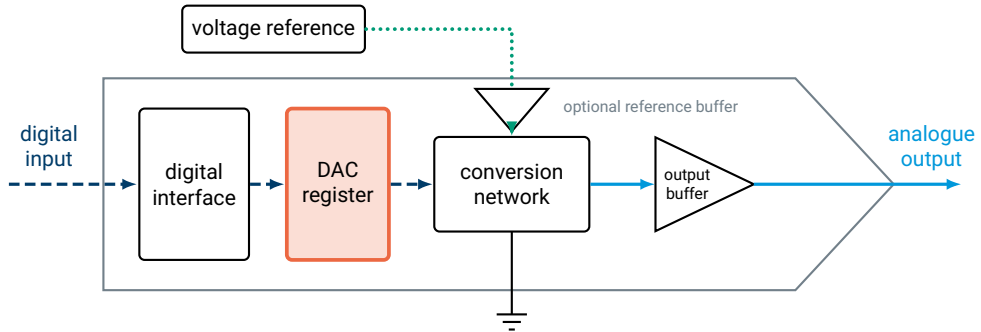
A DAC is a chain of functional blocks. We will step through them one at a time.

Building blocks of a DAC



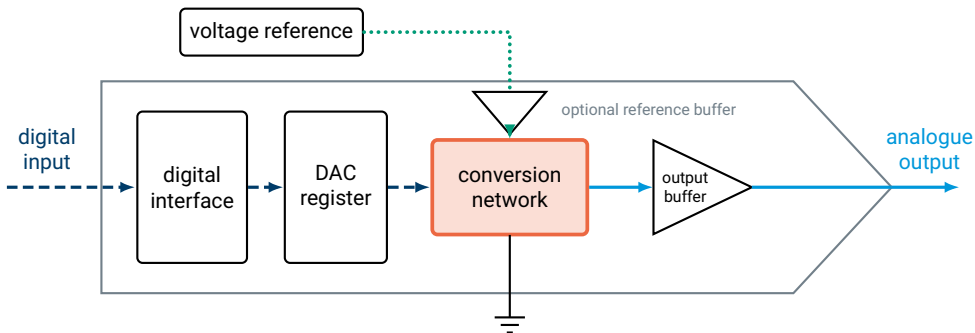
Digital interface: receives the code, for example over a serial bus or from a micro-controller register write.

Building blocks of a DAC



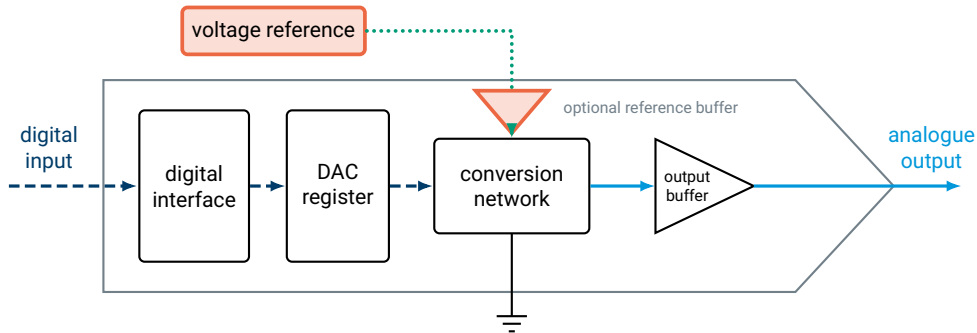
DAC register: holds the selected code until it is updated, so the output stays put between writes.

Building blocks of a DAC



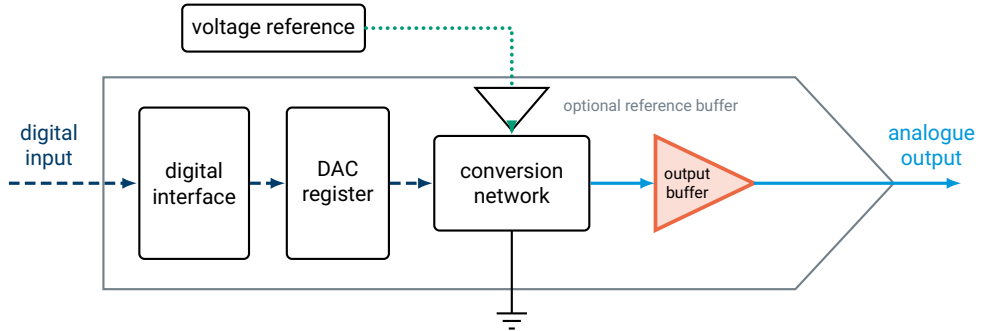
Conversion network: switches or combines analogue contributions according to the code. Its form depends on the architecture (Part 2).

Building blocks of a DAC



Reference: establishes the conversion scale; every output level is a fraction of it. An optional buffer stops the network loading it.

Building blocks of a DAC



Output buffer: where provided, isolates the conversion network from the load, within specified limits.

DAC basics: familiar ideas from ADCs



	Shared meaning	DAC interpretation
Resolution N	number of bits; N bits give 2^N possible codes	2^N distinct nominal output levels
Code D	the integer the binary word represents, 0 to $2^N - 1$	the input that selects which level is produced
Reference V_{REF}	sets the conversion scale	establishes the ideal output scale
LSB Δ	one step of the code	voltage difference between adjacent ideal output levels

Same vocabulary, opposite direction. A DAC produces the level a code names; it does not restore detail an ADC discarded.



Resolution, reference and one LSB



4 V is a convenient example value,
not the board's supply specification.

$$\Delta = \frac{V_{REF}}{2^N} = \frac{4\text{ V}}{16} = 0.25\text{ V}$$

1

$N = 4$: 16 levels, codes 0 to 15.

2

16 levels have only 15 increments, so the top level is $15\Delta = 3.75\text{ V}$, one LSB below V_{REF} .

3

Larger N at the same V_{REF} means finer spacing: $N = 8$ gives $\Delta \approx 15.6\text{ mV}$.

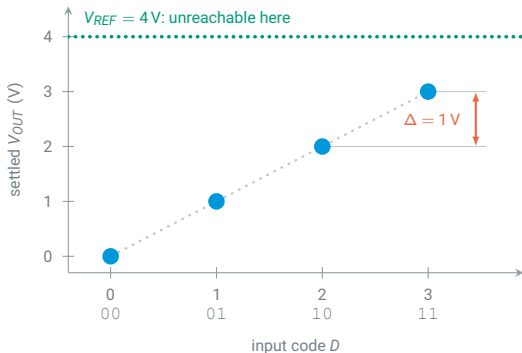
4

Resolution is not accuracy: small steps say nothing about how close each level is to its ideal value.



DAC transfer characteristic

$N = 2$, $V_{REF} = 4\text{ V}$, $\Delta = 1\text{ V}$: codes 00, 01, 10, 11 give 0, 1, 2, 3 V.



1

Last week's ADC plot: *voltage in, code out*. Here the quantities are reversed: *code in, voltage out*.

2

Only the four integer codes exist. The dotted line only guides the eye; there is no code 1.5.

3

This is the **code domain**, not time. The time-domain staircase comes later.



From code to voltage: the equation

$$\text{output} = (\text{number of increments}) \times (\text{size of one increment})$$



$$V_{OUT} = D \Delta = V_{REF} \frac{D}{2^N}$$

D integer code, 0 to $2^N - 1$

N resolution in bits

V_{REF} reference voltage, the scale

Δ one LSB, $V_{REF}/2^N$

Endpoint check: $D = 0$

$$V_{OUT} = 0 \times \Delta = 0 \text{ V}$$

Endpoint check: $D = 2^N - 1$

$$V_{OUT} = V_{REF} - \Delta; N = 4:$$
$$15 \times 0.25 = 3.75 \text{ V}$$



Worked example: code to voltage

$N = 4$, $V_{REF} = 4\text{ V}$, input code 1010_2 . What is V_{OUT} ?

bit	1	0	1	0
weight	8	4	2	1

Step	Value
------	-------

code value	
------------	--

one LSB	
---------	--

output	
--------	--



Worked example: code to voltage

$N = 4$, $V_{REF} = 4\text{ V}$, input code 1010_2 . What is V_{OUT} ?

bit	1	0	1	0
weight	8	4	2	1

Step	Value
code value	$D = 8 + 2 = 10$
one LSB	
output	



Worked example: code to voltage

$N = 4$, $V_{REF} = 4\text{ V}$, input code 1010_2 . What is V_{OUT} ?

bit	1	0	1	0
weight	8	4	2	1

Step	Value
code value	$D = 8 + 2 = 10$
one LSB	$\Delta = 4/16 = 0.25\text{ V}$
output	



Worked example: code to voltage

$N = 4$, $V_{REF} = 4\text{ V}$, input code 1010_2 . What is V_{OUT} ?

bit	1	0	1	0
weight	8	4	2	1

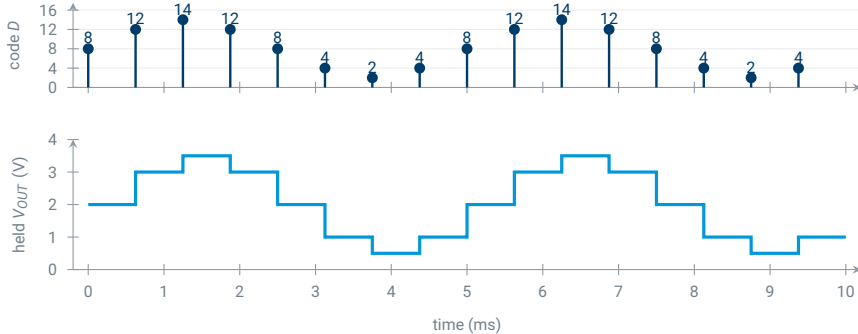
Step	Value
code value	$D = 8 + 2 = 10$
one LSB	$\Delta = 4/16 = 0.25\text{ V}$
output	$V_{OUT} = 10 \times 0.25 = \mathbf{2.50\text{ V}}$

Endpoint check: code 1111_2 gives $D = 15$, so $15 \times 0.25 = 3.75\text{ V}$, one LSB below V_{REF} .

An ideal DAC produces exactly the level its code names.



What comes out when the code changes with time?

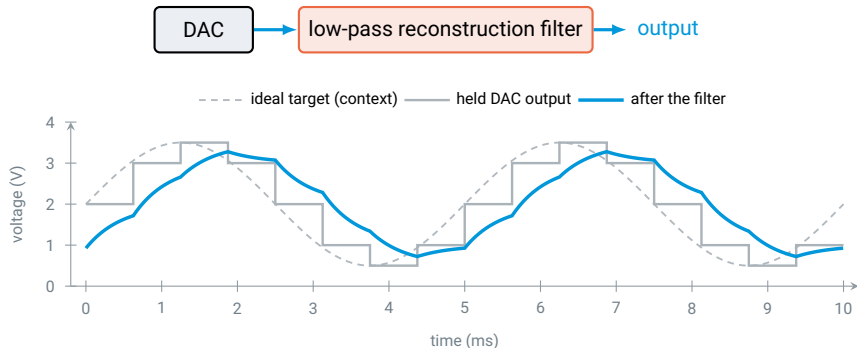


Codes 8, 12, 14, 12, 8, 4, 2, 4 give 2, 3, 3.5, 3, 2, 1, 0.5, 1 V, each *held* until the next update. At 1600 updates/s this 8-entry sequence repeats at $f_{signal} = f_{update}/M = 1600/8 = 200$ Hz.

Two separate controls: bit resolution sets the available levels; update timing sets when those levels change.



Why use a reconstruction filter?



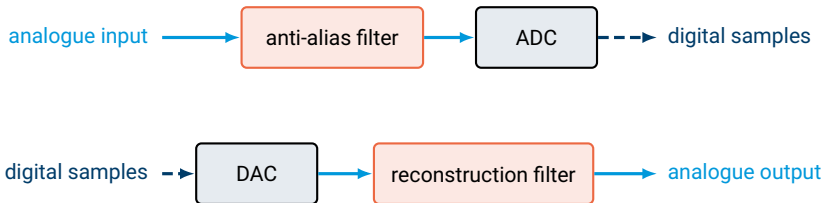
Holding and updating adds unwanted high-frequency content, including spectral images. The filter passes the wanted band and attenuates what lies above it.

Computed one-pole RC response (10 k Ω , 47 nF), steady state.

Filtering smooths, but it also attenuates and delays, and it cannot undo quantisation. This coarse example need not come out a perfect sine.



Reconstruction filtering and the ADC connection



Anti-alias filter

Before sampling: attenuates out-of-band input content that would otherwise alias into the wanted band.

Reconstruction filter

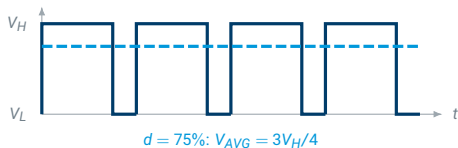
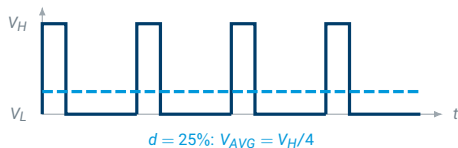
After the DAC: attenuates the unwanted high-frequency content that holding and updating add.

A filter after a DAC cannot repair aliasing that already happened during acquisition.



Next: how does the DAC actually generate its levels?

PWM-based DAC: cheap and simple

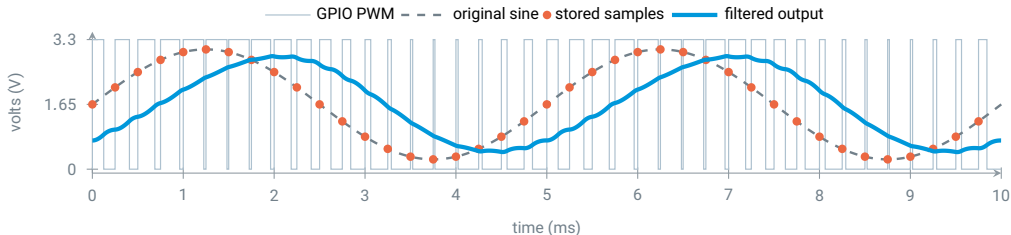


$$d = \frac{t_{on}}{T_{PWM}} \quad V_{AVG} = d V_H + (1 - d) V_L \quad V_L = 0 \Rightarrow V_{AVG} = d V_H$$

A low-cost way to approximate an analogue output without a dedicated multibit DAC. The GPIO pin itself still switches between just two levels.



Example: a 200 Hz sine from PWM



Remember: how closely the reconstructed sine matches the original sampled sine depends on the **quality of the filter** and on **how many samples** per period were stored.



Binary to a number: weight each bit

A DAC has to turn the bits into a level. The key idea: give each bit a **weight**, multiply by the bit, and **add**.

Three-bit example used for every architecture: $V_{REF} = 4\text{ V}$, $\Delta = 0.5\text{ V}$, code 101.

	b_2 (MSB)	b_1	b_0 (LSB)	
bit value	1	0	1	code 101
weight	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$	

$$D = 4b_2 + 2b_1 + b_0 = \sum_k b_k 2^k$$



Binary to a number: weight each bit

A DAC has to turn the bits into a level. The key idea: give each bit a **weight**, multiply by the bit, and **add**.

Three-bit example used for every architecture: $V_{REF} = 4\text{ V}$, $\Delta = 0.5\text{ V}$, code 101.

	b_2 (MSB)	b_1	b_0 (LSB)	
bit value	1	0	1	code 101
weight	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$	
bit $\times$ weight	4	0	1	$D = 4 + 0 + 1 = \mathbf{5}$

$$D = 4b_2 + 2b_1 + b_0 = \sum_k b_k 2^k$$

$$V_{OUT} = D \Delta = 5 \times 0.5 = \mathbf{2.5\text{ V}}$$



Binary to a number: weight each bit

A DAC has to turn the bits into a level. The key idea: give each bit a **weight**, multiply by the bit, and **add**.

Three-bit example used for every architecture: $V_{REF} = 4\text{ V}$, $\Delta = 0.5\text{ V}$, code 101.

	b_2 (MSB)	b_1	b_0 (LSB)	
bit value	1	0	1	code 101
weight	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$	
bit $\times$ weight	4	0	1	$D = 4 + 0 + 1 = \mathbf{5}$

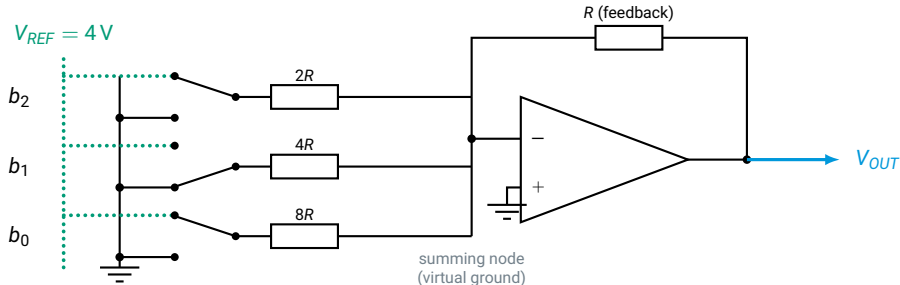
$$D = 4b_2 + 2b_1 + b_0 = \sum_k b_k 2^k \quad V_{OUT} = D \Delta = 5 \times 0.5 = \mathbf{2.5\text{ V}}$$

Times Δ , each bit is worth a fixed voltage: $b_2 \rightarrow 2\text{ V}$, $b_1 \rightarrow 1\text{ V}$, $b_0 \rightarrow 0.5\text{ V}$,
that is, $V_{OUT} = V_{REF} \left(\frac{b_2}{2} + \frac{b_1}{4} + \frac{b_0}{8} \right)$.

How can hardware give each bit its own weight, and add the results?



Binary-weighted resistor DAC

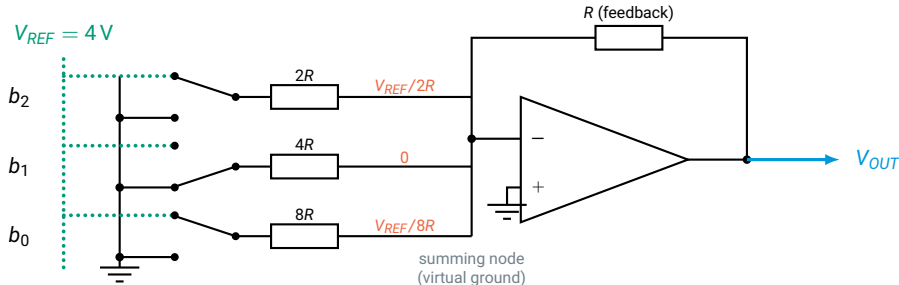


Each bit switches its **own** resistor to V_{REF} (1) or ground (0).

$$V_{OUT} = -R \left(\frac{b_2 V_{REF}}{2R} + \frac{b_1 V_{REF}}{4R} + \frac{b_0 V_{REF}}{8R} \right) = -V_{REF} \left(\frac{b_2}{2} + \frac{b_1}{4} + \frac{b_0}{8} \right) = -V_{REF} \frac{D}{8}$$



Binary-weighted resistor DAC



Each bit switches its **own** resistor to V_{REF} (1) or ground (0).

$$V_{OUT} = -R \left(\frac{b_2 V_{REF}}{2R} + \frac{b_1 V_{REF}}{4R} + \frac{b_0 V_{REF}}{8R} \right) = -V_{REF} \left(\frac{b_2}{2} + \frac{b_1}{4} + \frac{b_0}{8} \right) = -V_{REF} \frac{D}{8}$$

Code 101: $V_{OUT} = -4 \left(\frac{1}{2} + \frac{1}{8} \right) = -2.5 \text{ V}$; an inverter after it gives $+2.5 \text{ V} = D\Delta$.



The catch: a different resistor for every bit

Bits	Resistors	Spread	MSB accuracy
N	$2R \dots 2^N R$	$2^{N-1} : 1$	about $1/2^N$
3	$2R \dots 8R$	4:1	12.5 %
8	$2R \dots 256R$	128:1	0.4 %
12	$2R \dots 4096R$	2048:1	0.02 %
16	$2R \dots 65536R$	32768:1	0.002 %

1

Every bit needs its **own value**: N bits, N different resistors.

2

The MSB carries half of full scale, so its resistor must be accurate to about $1/2^N$.

3

Wide values are hard to make, match and keep matched over temperature; switch resistance hurts the small MSB resistors most.

MSB accuracy: the MSB resistor's error must keep its contribution within $\frac{1}{2}$ LSB.

The weighting idea is right; building many accurate resistor values is the problem, and it grows with resolution.



The catch: a different resistor for every bit

Bits	Resistors	Spread	MSB accuracy
N	$2R \dots 2^N R$	$2^{N-1} : 1$	about $1/2^N$
3	$2R \dots 8R$	4:1	12.5 %
8	$2R \dots 256R$	128:1	0.4 %
12	$2R \dots 4096R$	2048:1	0.02 %
16	$2R \dots 65536R$	32768:1	0.002 %

1

Every bit needs its **own value**: N bits, N different resistors.

2

The MSB carries half of full scale, so its resistor must be accurate to about $1/2^N$.

3

Wide values are hard to make, match and keep matched over temperature; switch resistance hurts the small MSB resistors most.

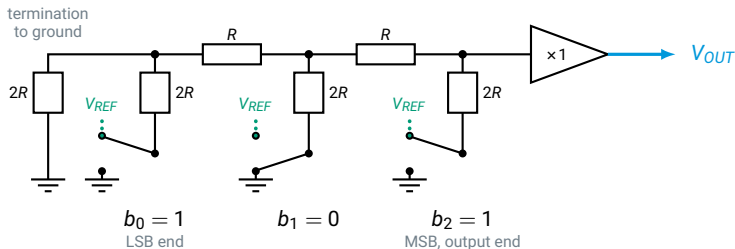
MSB accuracy: the MSB resistor's error must keep its contribution within $\frac{1}{2}$ LSB.

The weighting idea is right; building many accurate resistor values is the problem, and it grows with resolution.

*Can we keep the binary weighting, but use only **two** resistor values?*



R-2R ladder DAC



$$V_{OUT} = V_{REF} \left(\frac{b_2}{2} + \frac{b_1}{4} + \frac{b_0}{8} \right) = V_{REF} \frac{D}{8}$$

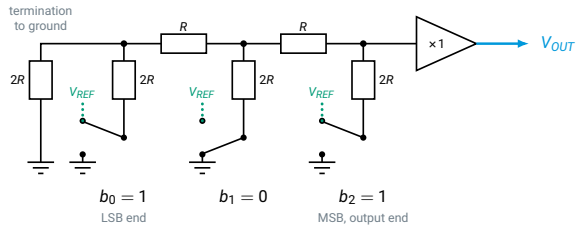
The **same weights** as the binary-weighted DAC, $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$ of V_{REF} , from only R and $2R$: each bit switches its $2R$ branch to V_{REF} (1) or ground (0).

Three-bit, non-inverting voltage-mode ladder into a high-impedance buffer, $V_{REF} = 4\text{ V}$. Shown holding 101.

Each step along the ladder halves a bit's contribution: binary weighting comes from repetition, not from a wide spread of resistor values.



R-2R worked example and trade-offs



$V_{REF} = 4\text{ V}$, code 101. What is V_{OUT} ?

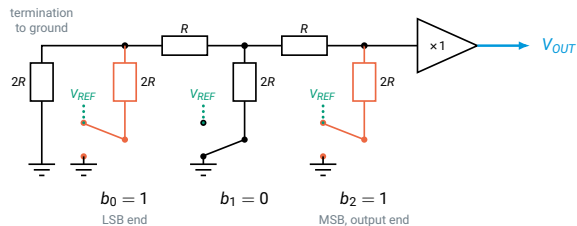
$b_2 = 1$
2.0 V

$b_1 = 0$
1.0 V

$b_0 = 1$
0.5 V



R-2R worked example and trade-offs



$V_{REF} = 4\text{ V}$, code 101. What is V_{OUT} ?

$$b_2 = 1 \\ \mathbf{2.0\text{ V}}$$

$$b_1 = 0 \\ 1.0\text{ V}$$

$$b_0 = 1 \\ \mathbf{0.5\text{ V}}$$

$$= 2 + 0.5 = \mathbf{2.5\text{ V}}$$

Check: $D = 5$, and $D\Delta = 5 \times 0.5 = 2.5\text{ V}$: the same output as the binary-weighted DAC.

Only two resistor values are needed, but ratio matching, switch resistance, loading and switching transients still matter.



Compare the approaches

Approach	Operating idea	Main advantage	Main limitation
PWM + filter	average a two-level pulse train set by duty d	uses a timer, no multibit DAC	ripple against response speed
Binary-weighted	one resistor per bit; sum the weighted currents	direct, one resistor per bit	$2^{N-1} : 1$ spread of values to match
R-2R ladder	add binary-weighted contributions	only two resistor values	ratio matching, switches, loading

All three realise the same ideal relationship $V_{OUT} = D \Delta$; they differ in how the levels are physically produced and in what limits them in practice.



Check your understanding

- a. Four bits, 4 V reference, code 1010: what is the output?
- b. Closest code for 2.70 V using the same DAC?
- c. Does a reconstruction filter recover information lost during quantisation?
- d. Which method encodes the desired average output using duty cycle?



Check your understanding

- a. Four bits, 4 V reference, code 1010: what is the output?
2.5 V ($D = 10, \Delta = 0.25 \text{ V}$)
- b. Closest code for 2.70 V using the same DAC?
- c. Does a reconstruction filter recover information lost during quantisation?
- d. Which method encodes the desired average output using duty cycle?



Check your understanding

- a. Four bits, 4 V reference, code 1010: what is the output?
2.5 V ($D = 10, \Delta = 0.25 \text{ V}$)
- b. Closest code for 2.70 V using the same DAC?
1011, giving **2.75 V** ($D^* = 10.8$, nearest is 11)
- c. Does a reconstruction filter recover information lost during quantisation?
- d. Which method encodes the desired average output using duty cycle?



Check your understanding

- a. Four bits, 4 V reference, code 1010: what is the output?
2.5 V ($D = 10, \Delta = 0.25 \text{ V}$)
- b. Closest code for 2.70 V using the same DAC?
1011, giving **2.75 V** ($D^* = 10.8$, nearest is 11)
- c. Does a reconstruction filter recover information lost during quantisation?
No. It smooths the output; the rounding already happened.
- d. Which method encodes the desired average output using duty cycle?



Check your understanding

- a. Four bits, 4 V reference, code 1010: what is the output?
2.5 V ($D = 10$, $\Delta = 0.25 \text{ V}$)
- b. Closest code for 2.70 V using the same DAC?
1011, giving **2.75 V** ($D^* = 10.8$, nearest is 11)
- c. Does a reconstruction filter recover information lost during quantisation?
No. It smooths the output; the rounding already happened.
- d. Which method encodes the desired average output using duty cycle?
PWM plus filtering.

The code selects amplitude, timing determines how the waveform evolves, and the implementation and filter determine the practical output behaviour.

